



I302 - Aprendizaje Automático y Aprendizaje Profundo

Trabajo Práctico 2: Clasificación y Ensemble Learning

Ignacio Carlos Basso

14 de abril de 2025

Ingeniería en Inteligencia Artificial

1. Diagnóstico de Cáncer de Mama

Resumen

En este trabajo se buscó modelar la probabilidad de que un tumor mamario sea maligno a partir de características celulares mediante **regresión logística binaria**. Para ello, se implementó desde cero dicho modelo con regularización L2, incorporando técnicas de rebalanceo, en caso de ser necesarias, para abordar el desbalance del dataset `cell_diagnosis_imbalanced_dev`. El modelo fue evaluado utilizando diversas métricas como el *F1-score*, comparando distintos valores del hiperparámetro de regularización y estrategias de balanceo como *undersampling*, *oversampling*, *SMOTE* y *cost re-weighting*. El modelo performó de manera óptima con un *F1-score* de 0.92 en el dataset balanceado y de 0.92 en el dataset desbalanceado con *cost re-weighting*, al utilizar una regularización L2 con $\lambda = 0,12$.

1.1. Introducción

El cáncer de mama es una de las principales causas de mortalidad femenina a nivel mundial, y su detección temprana resulta clave para mejorar la supervivencia. En este trabajo, se modela la probabilidad de que un tumor mamario sea maligno mediante **regresión logística binaria**, utilizando como entrada características celulares extraídas de imágenes histológicas de biopsias. Estas incluyen variables morfológicas y moleculares como tamaño y forma celular, densidad nuclear, textura de la cromatina, tamaño del citoplasma, adhesión celular, tasa de mitosis, estado de la membrana, concentración de factores de crecimiento, oxigenación, vascularización, marcadores inflamatorios, tipo celular y presencia de mutaciones genéticas.

Durante el análisis exploratorio se detectaron valores faltantes, atribuibles a diferencias entre los laboratorios que recolectaron los datos, así como *outliers* y rangos atípicos en algunas variables, aunque con baja correlación con la variable objetivo. El preprocesamiento incluyó: (i) detección y reemplazo de valores fuera del rango intercuartílico por NaNs, (ii) imputación de valores faltantes mediante *KNN* híbrido considerando variables numéricas y categóricas, (iii) codificación one-hot de variables categóricas y (iv) normalización de variables numéricas (excepto las binarias y la objetivo) usando media y desviación estándar

del conjunto de entrenamiento.

Finalmente, se construyó la matriz de predictores $\mathbf{X}$ a partir de las variables seleccionadas, a la cual se le agregó una columna de unos como término independiente (bias). El vector objetivo $\mathbf{y}$ corresponde a la variable “Diagnosis”, que indica si el tumor es benigno o maligno. Para abordar el desbalance presente en el segundo dataset, se implementaron distintas estrategias de balanceo.

1.2. Métodos

1.2.1. Regresión Logística Binaria

La **regresión logística** predice la probabilidad de que una observación pertenezca a una de dos clases, utilizando la función logística:

$$P(y = 1|X) = \frac{1}{1 + e^{-(\omega_0 + \omega_1 X_1 + \omega_2 X_2 + \dots + \omega_n X_n)}}$$

donde $P(y = 1|X)$ es la probabilidad de la clase 1 dada la característica X , y ω_j son los coeficientes a aprender.

El modelo se entrena minimizando la entropía cruzada regularizada, con reponderación de clases si es necesario:

$$J(\omega) = -\frac{1}{m} \sum_{i=1}^m w_{y^{(i)}} \left[y^{(i)} \log(\hat{y}^{(i)} + \epsilon) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)} + \epsilon) \right] + \frac{\lambda}{2m} \sum_{j=1}^n \omega_j^2$$

donde m es el número de muestras, $w_{y^{(i)}}$ el peso de la clase de $y^{(i)}$, λ es el coeficiente de regularización L2 y ϵ es un valor pequeño para evitar problemas numéricos.

El promedio y escalamiento del término de regularización aseguran que la pérdida sea comparable entre modelos y conjuntos de datos. La optimización se realiza mediante descenso de gradiente estocástico:

$$\omega_j := \omega_j - \alpha \frac{\partial J(\omega)}{\partial \omega_j}$$

donde α es la tasa de aprendizaje. La regularización L2 reduce el sobreajuste y la reponderación mejora el tratamiento de clases desbalanceadas. El modelo fue implementado desde cero considerando ambos aspectos y evaluado en el conjunto `cell_diagnosis_imbalanced_dev`.

1.2.2. K-Nearest Neighbor híbrido (KNN híbrido)

Para imputar valores faltantes se utilizó un algoritmo *KNN* que integra variables numéricas y categóricas. Inicialmente, se aplica una imputación básica: media para variables numéricas y moda para categóricas. Luego, las variables numéricas se normalizan (media cero, desvío estándar uno) y las categóricas se codifican como enteros.

La distancia entre observaciones se calcula combinando distancia euclidiana sobre variables numéricas y distancia de hamming sobre categóricas.

La distancia total se define como:

$$\text{Dist}(i, j) = \sqrt{\text{dist_euclidiana} + \lambda \cdot \text{dist_hamming}}$$

Para cada valor faltante, se seleccionan los k vecinos más cercanos según esta métrica, imputando la **media** si la variable es numérica o la **moda** si la variable es categórica.

El valor de k varía dinámicamente entre 3 y 15 según el tamaño del conjunto. En validación, la imputación se realiza usando exclusivamente información del conjunto de entrenamiento.

1.2.3. Normalización

Se utilizó la normalización Z-score para escalar las variables numéricas, transformando cada valor x en $z = \frac{x - \mu}{\sigma}$, donde μ es la media y σ la desviación estándar de la característica. Esta transformación, aplicada solo sobre las variables numéricas del conjunto de entrenamiento, asegura media cero y desviación estándar uno, mejorando la convergencia y el rendimiento del modelo sin inducir fuga de información.

1.2.4. Validación cruzada

Para seleccionar el mejor valor del hiperparámetro de regularización λ en **regresión logística** con penalización L2, se aplicó validación cruzada estratificada de k pliegues ($k = 5$ por defecto), optimizando el *F1-score*.

Dado un dataset $\mathcal{D}$, se divide en k subconjuntos disjuntos $\mathcal{D}_1, \dots, \mathcal{D}_k$. Para cada $\lambda \in \Lambda$, se entrena un modelo sobre $k - 1$ pliegues y se evalúa en el restante, obteniendo las predicciones $\hat{y}_j^{(\lambda)}$ y etiquetas reales y_j . Luego, se computa el *F1-score* global:

$$\text{F1}(\lambda) = \text{F1-score} \left(\bigcup_{j=1}^k \hat{y}_j^{(\lambda)}, \bigcup_{j=1}^k y_j \right) \\ \Rightarrow \lambda^* = \arg \max_{\lambda \in \Lambda} \text{F1}(\lambda)$$

Cada iteración incluye el mismo pipeline: (i) tratamiento de *outliers* con IQR, (ii) imputación con KNN híbrido, (iii) codificación *one-hot*, (iv) normalización de variables numéricas.

cas (excepto binarias y *target*), y agregado del término independiente. También puede aplicarse reponderación o remuestreo en el conjunto de entrenamiento.

El modelo final se entrena con el mejor λ^* sobre todo el conjunto de entrenamiento.

1.2.5. Descripción del conjunto de datos y preprocesamiento

El dataset `cell_diagnosis_dev` se utilizó para el desarrollo del modelo, reservando `test` para evaluación final. Dentro de `dev`, se dividió un 80 % para entrenamiento y 20 % para validación. Este esquema se aplicó en ejercicios como el 1.2 (entrenamiento y validación con `dev`) y 1.3 (evaluación con `test`), así como en el 1.4 y 1.5 con el conjunto `cell_diagnosis_imbalanced_dev`.

El preprocesamiento incluyó:

- Detección de *outliers* mediante IQR (usado para detectar y corregir valores atípicos en variables numéricas) y reemplazo por NaN.
- Imputación con KNN híbrido para variables numéricas y categóricas.
- Codificación *one-hot* de variables categóricas.
- Normalización del conjunto (ver sección 1.2.3).
- Construcción de matrices **X** e **y** con `prepare_data`, agregando el término de sesgo.

Todas las transformaciones (IQR, imputación y normalización) se basaron únicamente en el conjunto de entrenamiento, evitando fuga de información. El resultado del preprocesamiento puede visualizarse en la figura A.2.

En el ejercicio 1.4, se aplicaron técnicas para abordar el desbalance de clases: *undersampling*, *oversampling* por duplicación,

SMOTE y *cost re-weighting*, esta última ajustando la función de pérdida sin modificar los datos directamente.

1.2.6. Técnicas de rebalanceo

En el ejercicio 1.4 se aplicaron distintas estrategias para mitigar el desbalance de clases y evitar que el modelo se sesgue hacia la clase mayoritaria. Las técnicas implementadas fueron:

- **Undersampling**: eliminación aleatoria de instancias de la clase mayoritaria hasta igualar la cantidad de la clase minoritaria. Reduce el tamaño total del conjunto, pero evita generar datos sintéticos.
- **Oversampling por duplicación**: replicación aleatoria de muestras de la clase minoritaria hasta alcanzar la cantidad de la clase mayoritaria, incrementando el tamaño del conjunto y reforzando sus patrones para aprender mejor las características de la clase minoritaria.
- **SMOTE (Synthetic Minority Oversampling Technique)**: genera muestras sintéticas de la clase minoritaria interpolando con sus vecinos más cercanos, aportando variabilidad adicional y mejorando la generalización.
- **Cost re-weighting**: ajusta la función de costo para penalizar más los errores sobre la clase minoritaria, multiplicando su contribución por un factor $C = \frac{\pi_2}{\pi_1}$, siendo π_1 y π_2 las probabilidades a priori de las clases minoritaria y mayoritaria, respectivamente.

Se compararon los efectos de cada técnica sobre el rendimiento del modelo en un conjunto desbalanceado, utilizando métricas como *F1-score*, y curvas ROC y PR.

1.2.7. Selección de hiperparámetros

Para ajustar el hiperparámetro de regularización λ en la **regresión logística** L2, se aplicaron tres enfoques:

1. **Barrido simple:** evaluación de 20 valores logarítmicamente espaciados entre 10^{-4} y 10^3 con `np.logspace`, seleccionando el que maximizó el *F1-score* sobre el conjunto de validación.
2. **Barrido refinado:** luego de identificar los dos mejores λ del barrido simple, se generaron 10 valores equiespaciados entre ellos para una búsqueda más precisa.
3. **Validación cruzada estratificada:** evaluación de cada λ mediante validación cruzada de k pliegues ($k = 5$ por defecto), seleccionando el que obtuvo mayor *F1-score* promedio.

Los tres métodos produjeron desempeños equivalentes. Por eficiencia computacional, se priorizó el uso de barridos simple y refinado, descartando la validación cruzada. En presencia de *cost re-weighting*, se mantuvo la misma grilla de λ incorporando los pesos de clase en la función de costo.

Cuadro 1: Hiperparámetros obtenidos para cada técnica (Punto 1.2).

Técnica	λ óptimo
Sin rebalanceo	0.0001
Undersampling	0.0001
Oversampling duplicado	0.0001
SMOTE	0.0016
Cost re-weighting	0.1255

1.2.8. Métricas de performance

Las métricas de performance utilizadas para evaluar el rendimiento del modelo de **regresión logística binaria** incluyen aquellas basadas en la *matriz de confusión*, como *accuracy*, *precision*, *recall* y *F1-score*, todas

ellas computadas tanto por clase como con promedio macro. Además, se utilizaron métricas basadas en curvas, como la *curva ROC* y la *curva Precision-Recall* (PR), junto con sus respectivas áreas bajo la curva: *AUC-ROC* y *AUC-PR*. La Tabla 3 resume las fórmulas utilizadas para cada una de estas métricas.

1.3. Resultados

En el punto 1.2, con el conjunto de datos de desarrollo `cell_diagnosis_dev`, se obtuvo un *F1-score* de 0.93 (ver cuadro A.4), lo que refleja un buen rendimiento del modelo. El valor de $\lambda = 0,0001$ para la regularización L2 sugiere que el modelo no está sobreajustado y mantiene una adecuada capacidad de generalización.

El buen desempeño del modelo se refleja en las curvas ROC y PR de la Figura 1. Por otro lado, en el apéndice A.3 se presenta la evolución del *F1-score* en función del valor del hiperparámetro λ , donde se observa que el valor óptimo es $\lambda = 0,0001$.

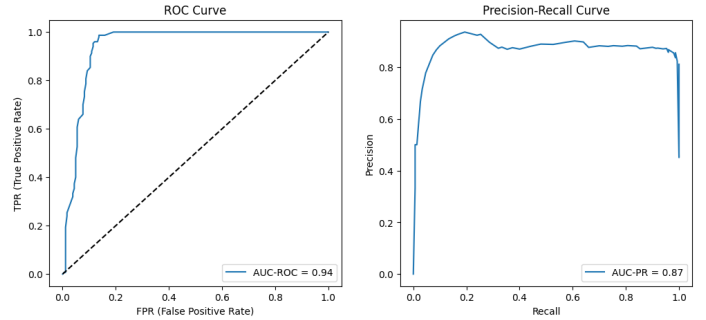


Figura 1: Curvas ROC y PR del modelo en el conjunto de datos de validación.

La curva ROC muestra que el modelo tiene una excelente capacidad de discriminación, con un *AUC-ROC* de 0.94. Esto indica que logra mantener una alta tasa de *true positives* (TPR) mientras controla la tasa de *false positives* (FPR), lo cual es evidencia de un buen desempeño general al distinguir entre clases positivas y negativas.

Por otro lado, la curva *Precision-Recall* (PR) presenta un *AUC-PR* de 0.87, lo que también sugiere un buen rendimiento, especialmente en contextos con clases desbalanceadas. Sin embargo, se observa una caída abrupta de la *precision* cuando el *recall* se aproxima a 1. Esta caída se debe al aumento de *false positives* (FP) cuando el modelo se vuelve más permisivo al clasificar instancias como positivas, lo que reduce la proporción de *true positives* entre las predicciones positivas. Esta pérdida de *precision* refleja el costo de priorizar la recuperación de positivos.

Al evaluar el modelo en el conjunto de prueba `cell_diagnosis_test` (punto 1.3), se obtuvo un *F1-score* de 0.92, confirmando que el modelo mantiene un buen rendimiento en datos no vistos. Las métricas obtenidas son:

Cuadro 2: Métricas el modelo en el conjunto de test.

Métrica	Valor
Accuracy	0.91
Precision	0.92
Recall	0.92
F1-score	0.92
AUC ROC	0.94
AUC PR	0.89

Las curvas ROC y PR muestran un *AUC* de 0.94 y 0.89, respectivamente, lo cual indica que el modelo generaliza bien. Logrando apreciar en la Figura A.5 que se presenta un comportamiento muy similar al de la Figura 1, teniendo ambas una caída al tener un elevado *recall*.

En resumen, el modelo de **regresión logística binaria** con un dataset balanceado obtuvo un *F1-score* de 0.93 en el conjunto de desarrollo y 0.92 en el conjunto de prueba, evidenciando que el modelo generaliza bien a datos no vistos sin sobreajustarse. Esto se debe a la correcta selección de hiperparámetros, la regularización L2 y el preprocesamiento adecuado, que incluyó imputación de valores faltantes y normalización.

Al trabajar con un conjunto de datos des-

balanceado, se probaron diversas técnicas de rebalanceo. En la Tabla A.6, se compararon cinco enfoques: sin rebalanceo, *undersampling*, *oversampling* por duplicación, *oversampling* mediante *SMOTE* y *cost re-weighting*. Cada modelo se entrenó con el valor óptimo de λ , obtenido mediante un barrido sistemático.

En el conjunto de validación, el modelo con *cost re-weighting* presentó el mejor desempeño general al analizar las métricas en su conjunto. Obtuvo la mayor *accuracy* (0.87), junto con un excelente *recall* (0.90) y un competitivo *F1-score* (0.85). Este enfoque logra un buen equilibrio entre precisión y sensibilidad, lo cual es especialmente importante en contextos con clases desbalanceadas.

Sin embargo, el modelo *sin rebalanceo* mostró métricas muy similares, incluyendo un *F1-score* idéntico (0.85) y un rendimiento comparable en *precision*, *recall* y *AUC ROC*. Dado que este modelo no introduce modificaciones en la función de pérdida ni genera datos adicionales, es más simple y puede ser preferido cuando se busca un modelo menos complejo sin comprometer la performance.

En contraste, el modelo con *SMOTE* presentó el peor rendimiento, con el menor *F1-score* (0.82) y la menor *precision* (0.77), sumado a una mayor complejidad debido a la generación de instancias sintéticas, lo que lo vuelve menos recomendable.

Siendo importante visualizar el comportamiento descrito en la Figura 5 (ver en A.7 para mayor detalle), donde se observan las curvas ROC y PR de los modelos en el conjunto de validación desbalanceado. En el caso de los dos modelos recomendables sus curvas ROC muestran un *AUC-ROC* de 0.93, lo que indica una buena capacidad de discriminación del modelo. La curva PR, con un *AUC-PR* de 0.68 y 0.67, también sugiere un rendimiento aceptable, aunque con una caída abrupta en la *precision* a medida que el *recall* aumenta, sucediendo en todos los modelos de igual forma que en las figuras 1 y A.5. Es posible apreciar

que ante un *recall* elevado *cost re-weighting* sufre una caída, pero luego recupera la *precision*, lo que indica que el modelo es capaz de recuperar la *precision* al clasificar instancias como positivas, lo que reduce la proporción de *true positives* entre las predicciones positivas. Esto sugiere que el modelo tiene un buen equilibrio entre sensibilidad y precisión, lo cual es deseable en problemas de clasificación desbalanceada.

En el conjunto de prueba, el modelo con *cost re-weighting* alcanzó el mejor *F1-score* (0.92) y el mayor *recall* (0.95), acompañado por una alta *accuracy* (0.93) y un excelente *AUC ROC* (0.96). Esto confirma que el modelo logra una muy buena capacidad de generalización, manteniendo un equilibrio entre sensibilidad y precisión (ver Tabla 6).

El modelo *sin rebalanceo* también mostró un rendimiento muy competitivo, con un *F1-score* de 0.91, una *accuracy* igual (0.93) y métricas similares en el resto de los indicadores. Si bien su *recall* fue levemente menor, sigue siendo una opción muy sólida. Esto se

debe a que las clases presentan una alta separabilidad en el espacio de *features*, lo que permite buenos resultados incluso sin aplicar técnicas específicas de rebalanceo. Además, al no introducir modificaciones adicionales, su simplicidad puede hacerlo preferible cuando se desea una solución más interpretable o de menor complejidad.

Por otro lado, *SMOTE* volvió a presentar el peor rendimiento relativo: a pesar de un buen *AUC ROC* (0.96), su menor *precision* (0.80) y su menor *accuracy* (0.84) lo posicionan como la opción menos efectiva, especialmente si se considera la complejidad agregada por la generación de datos sintéticos.

En resumen, tanto *cost re-weighting* como *sin rebalanceo* ofrecen resultados sobresalientes. La elección entre ambos dependerá del balance que se quiera alcanzar entre performance y simplicidad del modelo, siendo razonable preferir el modelo sin rebalanceo cuando se prioriza una implementación más sencilla sin pérdida significativa de calidad.

2. Predicción de Rendimiento de Jugadores de Basketball

Resumen

En este trabajo se buscó desarrollar modelos predictivos para estimar la probabilidad de que un jugador de baloncesto pertenezca a una de las tres clases definidas en la columna `war_class`: Negative WAR, Null WAR y Positive WAR. Para llevarlo a cabo, se realizaron análisis exploratorios sobre los datos del archivo `WAR_class_dev.csv` para determinar el preprocesamiento adecuado, aplicando un codificado de etiquetas para convertir las clases a enteros consecutivos. Se implementaron tres modelos: Análisis Discriminante Lineal (LDA), **regresión logística** regularizada con softmax para clasificación multiclase y **Random Forest** utilizando entropía como criterio de división. Después de entrenar los modelos con `WAR_class_dev`, se evaluaron con el conjunto `WAR_class_test`. Los resultados mostraron que **Random Forest** fue el modelo con el mejor rendimiento, alcanzando un *accuracy* de 0.94, *precision* de 0.94, *recall* de 0.95, *F1-score* de 0.94, *AUC-ROC* de 0.99 y *AUC-PR* de 0.98.

2.1. Introducción

La predicción del impacto de un jugador de baloncesto sobre el rendimiento de su equipo es una tarea clave en el análisis deportivo moderno. En este contexto, se busca construir

un clasificador que, a partir de ciertas métricas individuales, determine a cuál de las tres categorías definidas por la variable `war_class` pertenece un jugador.

Las variables disponibles incluyen un con-

junto de métricas individuales de cada jugador, tales como `poss` (posesiones), `mp` (minutos jugados), `raptor_total` (impacto total estimado por el modelo RAPTOR), `war_total` (valor continuo de WAR) y `pace_impact` (impacto del jugador en el ritmo de juego). La variable de salida del modelo, `war_class`, es categórica y representa el nivel de contribución del jugador con tres posibles clases.

Para abordar esta tarea, se implementaron tres algoritmos de clasificación: Análisis Discriminante Lineal (**LDA**), **regresión logística** regularizada multiclase con *softmax* y **Random Forest** con entropía como criterio de división. Cada uno de estos modelos fue entrenado y evaluado utilizando los conjuntos `WAR_class_dev` y `WAR_class_test`, con el objetivo de comparar su capacidad predictiva en términos de métricas relevantes para clasificación multiclase.

2.2. Métodos

Los métodos de preprocesamiento utilizados en este trabajo —el método del IQR, el *KNN* híbrido (1.2.2), el *One Hot Encoding* y la normalización (1.2.3) — fueron descriptos previamente y se continúan empleando; pueden consultarse en las secciones previas.

2.2.1. Regresión Logística Multiclase

La **regresión logística multiclase** extiende el modelo logístico a problemas con más de dos clases. En lugar de utilizar una función sigmoide, se emplea la función *softmax* para modelar la probabilidad de pertenencia a cada clase:

$$p(C_j | X) = \hat{y}_j(X, w) = \frac{e^{\phi^T(x)w_j}}{\sum_{k=1}^K e^{\phi^T(x)w_k}} \quad (1)$$

donde K es el número total de clases, w_j es el vector de parámetros asociado a la clase C_j , y $\phi(x)$ representa la transformación (posiblemente no lineal) aplicada al vector de entrada

x . Esta formulación permite asignar una probabilidad a cada clase, garantizando que la suma total sea 1.

El entrenamiento se realiza minimizando la entropía cruzada entre las distribuciones predicha y verdadera. Este modelo es una generalización natural del presentado en la Sección 1.2.1, y conserva sus ventajas en términos de interpretabilidad y eficiencia computacional.

2.2.2. Análisis Discriminante Lineal (LDA)

El Análisis Discriminante Lineal (**LDA**) es un método supervisado de clasificación que busca combinaciones lineales de variables que separen mejor las clases. Supone que los datos provienen de distribuciones normales multivariadas con una matriz de covarianza común Σ .

El objetivo es maximizar la razón entre la varianza entre clases y la varianza dentro de las clases, resolviendo el problema de autovectores de $S_W^{-1}S_B$, con S_B y S_W como matrices de covarianza entre y dentro de las clases.

Se implementó **LDA** desde cero, estimando Σ como promedio ponderado de las covarianzas por clase. La clasificación se basó en funciones discriminantes lineales:

$$\delta_k(x) = x^T \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^T \Sigma^{-1} \mu_k + \log \pi_k$$

donde μ_k es la media de la clase k y π_k su probabilidad a priori. Estas funciones provienen de la log-verosimilitud de una gaussiana multivariada:

$$\ln(p(C_k | x)) = \ln(\pi_k) - \frac{1}{2} \ln |2\pi\Sigma| - \frac{1}{2} (x - \mu_k)^T \Sigma^{-1} (x - \mu_k)$$

Omite términos constantes para clasificar. Además, se aplicó *softmax* a los scores para estimar probabilidades.

2.2.3. Random Forest

El **Random Forest** es un algoritmo de clasificación y regresión basado en un conjunto de árboles de decisión (cada uno entrenado con una muestra aleatoria del conjunto de datos original), diseñado para mejorar la precisión y reducir el sobreajuste combinando múltiples predictores.

Su construcción sigue estos pasos:

1. **Bagging:** Se generan subconjuntos del conjunto de datos mediante muestreo con reemplazo (bootstrap), y cada árbol se entrena sobre uno distinto.
2. **Construcción de árboles:** En cada nodo, se selecciona aleatoriamente un subconjunto de variables para elegir la mejor división según un criterio como la entropía o el índice Gini.
3. **Predicción:** Para clasificación, cada árbol vota y se elige la clase mayoritaria; para regresión, se promedian las predicciones.

El **Random Forest** es robusto frente al ruido, al sobreajuste y a características irrelevantes. También permite estimar la importancia de cada variable. Sin embargo, su interpretabilidad es inferior a la de un solo árbol de decisión.

2.2.4. Label Encoding

Se aplicó *Label Encoding* para convertir la variable categórica `war_class` en valores numéricos, asignando **Negative WAR: 0**, **Null WAR: 1** y **Positive WAR: 2**. Esto permitió utilizarla como *target* en modelos como **regresión logística** o **Random Forest**, que requieren entradas numéricas. Si bien las clases no tienen un orden natural, se consideró esta codificación adecuada dadas las características del problema.

2.2.5. Validación cruzada

La validación cruzada es una técnica fundamental para estimar el rendimiento generalizable de un modelo y comparar combinaciones de hiperparámetros de forma robusta. En este trabajo se implementó validación cruzada estratificada con $k = 5$ pliegues.

Para cada combinación de hiperparámetros, se entrenó el modelo en 4 de los 5 *folds* y se lo evaluó en el restante, repitiendo este proceso de forma rotativa. En lugar de promediar las métricas individuales por pliegue, se acumularon todas las predicciones y etiquetas reales, y se calculó un único *F1-score macro* sobre la concatenación de ambos conjuntos. Esto brindó una estimación más representativa del desempeño global del modelo.

Este enfoque evitó el sesgo asociado a una única partición de los datos y permitió una comparación justa entre configuraciones, facilitando la selección del modelo óptimo.

2.2.6. Descripción del conjunto de datos y preprocesamiento

El conjunto `WAR_class_dev` fue dividido en 80 % para entrenamiento y 20 % para validación (ejercicio 2.3), mientras que en el ejercicio 2.4 se entrenó con todo el conjunto `dev` y se evaluó con `WAR_class_test`.

El preprocesamiento incluyó:

- Eliminación de variables con alta información mutua respecto a `war_class`, como `war_total`, para evitar redundancia (ver apéndice A.9). En particular, `war_total` fue descartada por ser una versión continua del *target* discretizado, lo cual implicaría una fuga directa de información (*data leakage*).
- Detección de valores atípicos mediante IQR y reemplazo por NaN.
- Imputación de faltantes con KNN híbrido, aplicada tanto a variables numéricas como categóricas.

- Codificación de variables categóricas mediante *one-hot encoding*.
- Normalización de todas las características para unificar escalas.
- Aplicación de `LabelEncoder` sobre y , transformando etiquetas a valores enteros consecutivos.
- Generación de las matrices $\mathbf{X}$ e $\mathbf{y}$ con `prepare_data`, incorporando término de sesgo en todos los modelos salvo en **LDA**, donde no se añade ya que el algoritmo considera directamente las distribuciones de probabilidad de las clases y asume que los datos ya están centrados, por lo que agregarlo no tendría sentido en ese contexto.

Todos los cálculos de IQR, imputación y normalización se realizaron exclusivamente con datos del conjunto de entrenamiento, para evitar fugas de información. La técnica de normalización utilizada se describe en la sección 1.2.3.

2.2.7. Selección de hiperparámetros

Para el modelo de **regresión Logística** con regularización L2 multiclase se realizó una búsqueda sistemática del hiperparámetro λ , del mismo modo que en la Sección 1.2.7, resultando como valor óptimo $\lambda = 0,0379$ (puede verse el gráfico *F1-score* vs λ en el apéndice A.10). En el caso del modelo de **Random Forest**, se aplicó validación cruzada con 5 *folds* para seleccionar los mejores hiperparámetros. Se exploraron los siguientes valores: $n_estimators \in \{5, 10, 20\}$, $max_depth \in \{3, 4, 6\}$, $min_samples_split \in \{2, 5\}$, y $sample_ratio \in \{0,6, 0,8\}$. Estos parámetros controlan respectivamente la cantidad de árboles, la profundidad máxima de cada árbol, el número mínimo de muestras requeridas para dividir un nodo, y la proporción de muestras utilizadas para construir cada árbol.

El mejor rendimiento se obtuvo con $n_estimators = 10$, $max_depth = 6$, $min_samples_split = 5$ y $sample_ratio = 0,8$. Estos hiperparámetros fueron seleccionados en función de su capacidad para maximizar el *F1-score* en el conjunto de validación, lo que indica un buen equilibrio entre *precision* y *recall*.

2.2.8. Extensión de métricas al caso multiclase

Las métricas binarias (sección 1.2.8) se extendieron al caso multiclase mediante el enfoque *uno contra el resto*, calculando *precision*, *recall*, *F1-score*, *AUC-ROC* y *AUC-PR* por clase y promediando con:

$$F1_{\text{macro}} = \frac{1}{C} \sum_{c=1}^C F1_c$$

La *accuracy* se mantuvo sin cambios y la matriz de confusión se generalizó a $C \times C$. Las métricas fueron implementadas desde cero, usando regla del punto derecho para aproximar áreas bajo curva. La Tabla 3 resume todas las fórmulas.

2.3. Resultados

En el punto 2.3 se entrenaron los modelos de **LDA** (2.2.2), **regresión logística multiclase con regularización L2** (2.2.1) y **Random Forest** (2.2.3) sobre el conjunto de entrenamiento, y se evaluaron en el conjunto de validación. Las métricas obtenidas se presentan en la Tabla A.11.

Según los resultados, **Random Forest** fue el modelo con mejor desempeño, alcanzando los valores más altos en todas las métricas (*accuracy*, *precision*, *recall*, *F1-score*, *AUC-ROC* y *AUC-PR*), lo que indica un excelente equilibrio entre sensibilidad y especificidad, logrando una clasificación precisa de tanto los casos positivos como los negativos, con alta confianza. En contraste, la **regresión logística multiclase** obtuvo el menor rendimiento,

aunque junto con **LDA** mostró un desempeño competitivo pese a su simplicidad.

Si bien **Random Forest** fue claramente superior, modelos como **LDA** (con un *F1-score* de 0.89 frente al 0.94 de **Random Forest**) pueden ser preferibles en escenarios donde se prioricen la simplicidad, la interpretabilidad y la eficiencia computacional.

Las figuras 8, 9 y 10 ilustran el desempeño de los modelos. En **Random Forest**, se observa que alcanza un *AUC ROC* de 0.99 y un *AUC PR* de 0.98 en la mayoría de sus clases, lo cual indica un rendimiento excepcional en la clasificación multiclase. Además, las curvas *Precision-Recall* (PR) presentan una forma homogénea entre clases: comienzan aproximadamente en el punto (0, 1) y finalizan en (1, 0). Este comportamiento uniforme se debe a que **Random Forest** produce distribuciones de probabilidad discretizadas y poco calibradas, resultado de la votación entre árboles. En consecuencia, los valores de probabilidad tienden a concentrarse cerca de 0 o 1, generando curvas PR más suaves y con la trayectoria esperada.

Por otro lado, **LDA** presenta mayor variabilidad entre clases, siendo la clase 2 la de peor rendimiento. Esto se evidencia al observar que su curva ROC se aproxima más a la diagonal, lo que indica una menor capacidad discriminativa.

A diferencia de **Random Forest**, tanto **LDA** como **regresión logística multiclase** generan probabilidades calibradas y continuas, que permiten reflejar de forma más precisa la incertidumbre del modelo. En estos casos, se observa que la clase 1 presenta una curva PR con comportamiento más abrupto: inicia cerca de (0, 0), asciende a un punto próximo a (0.98, 0.9) y luego desciende hacia (1, 0.3). Este patrón indica que el modelo identifica con alta certeza un subconjunto pequeño de instancias positivas (alta *precision*

y buen *recall* inicial), pero al relajar el umbral de decisión, se incorporan numerosos *false positives*, lo que deteriora la *precision*.

Este fenómeno puede explicarse por una combinación de factores: dificultad del modelo para separar correctamente esa clase en el espacio de características, posible desbalance en su frecuencia o una mayor superposición con otras clases. A diferencia de **Random Forest**, los modelos lineales como **LDA** y **regresión logística** son más sensibles a estas variaciones, lo cual se refleja en la forma de sus curvas PR.

En el punto 2.4 se reentrenaron todos los modelos utilizando el conjunto completo de desarrollo `WAR_class_dev` y se evaluaron sobre el conjunto de prueba `WAR_class_test`. Las métricas obtenidas se resumen en la Tabla 8.

Los resultados en el conjunto de prueba reafirman las conclusiones anteriores. **Random Forest** fue nuevamente el modelo con mejor rendimiento, alcanzando valores cercanos a 0.94 o superiores en todas las métricas. Esto confirma su capacidad para generalizar correctamente y mantener un buen desempeño más allá del conjunto de validación.

La **regresión logística** volvió a ubicarse por debajo de los otros dos modelos, mientras que **LDA** logró una performance competitiva, destacándose especialmente en *precision*, *recall* y *AUC ROC*. Dado que **LDA** es considerablemente más simple que **Random Forest**, podría preferirse en situaciones donde la diferencia de rendimiento no justifique el uso de modelos más complejos.

En conclusión, **Random Forest** fue el modelo con mejor capacidad predictiva tanto en validación como en prueba. No obstante, **LDA** mostró ser una alternativa simple y eficaz, especialmente en escenarios donde la interpretabilidad o los recursos computacionales limitados son factores determinantes.

A. Apéndice

A.1. Métricas de performance

Cuadro 3: Resumen de métricas de evaluación en clasificación binaria y multiclase.

Concepto	Fórmula
Matriz de confusión	$\begin{bmatrix} \text{TP} & \text{FP} \\ \text{FN} & \text{TN} \end{bmatrix}$
Accuracy	$\frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{FN} + \text{TN}}$
Precision (por clase)	$\text{Precision}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c}$
Precision (macro)	$\text{Precision}_{\text{macro}} = \frac{1}{C} \sum_{c=1}^C \text{Precision}_c$
Recall (por clase)	$\text{Recall}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FN}_c}$
Recall (macro)	$\text{Recall}_{\text{macro}} = \frac{1}{C} \sum_{c=1}^C \text{Recall}_c$
F1-score (por clase)	$\text{F1}_c = \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}$
F1-score (macro)	$\text{F1}_{\text{macro}} = \frac{1}{C} \sum_{c=1}^C \text{F1}_c$
Curva ROC	$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad \text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$
AUC ROC	$\text{AUC ROC} = \int_0^1 \text{TPR}(x) dx$
Curva PR	$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$
AUC PR	$\text{AUC-PR} = \int_0^1 \text{Precision}(r) dr$

A.2. Visualización del preprocesamiento en el 1.1

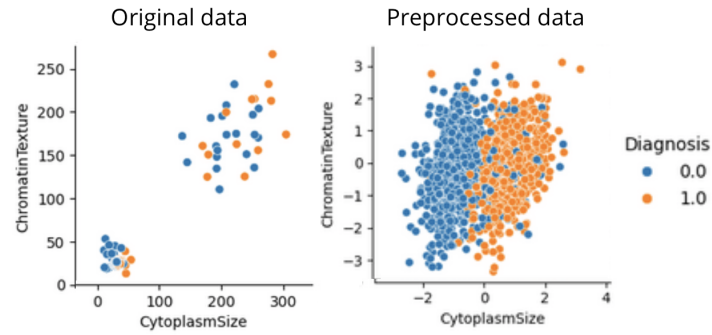


Figura 2: Distribución de los features en el dataset luego del preprocesamiento.

A.3. F1-score vs λ del ejercicio 1.2

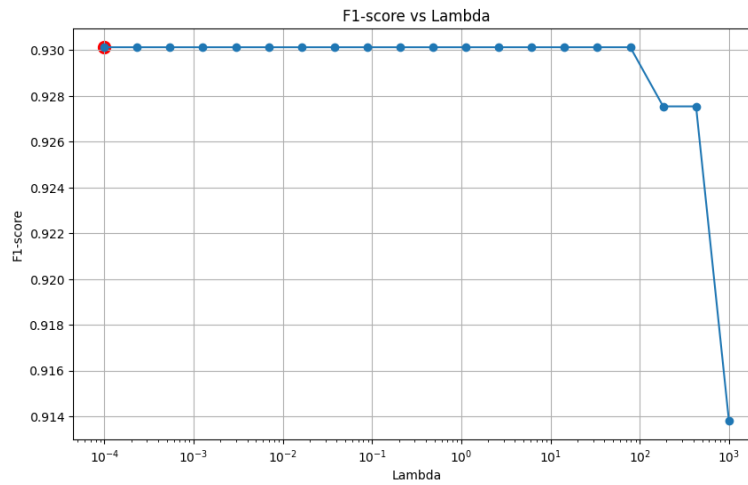


Figura 3: Evolución de λ vs F1-score utilizando un único barrido.

Es posible apreciar en la figura 3 que el valor óptimo de λ se encuentra en torno a 0.0001, ya que a partir de este valor la métrica comienza a decrecer. Siendo este el punto donde se obtiene el mayor *F1-score*.

A.4. Métricas de performance del ejercicio 1.2

Cuadro 4: Métricas del modelo en el conjunto de validación.

Métrica	Accuracy	Precision	Recall	F1-score	AUC ROC	AUC PR
Valor	0.92	0.93	0.93	0.93	0.94	0.87

A.5. Curvas AUC ROC y AUC PR del ejercicio 1.3

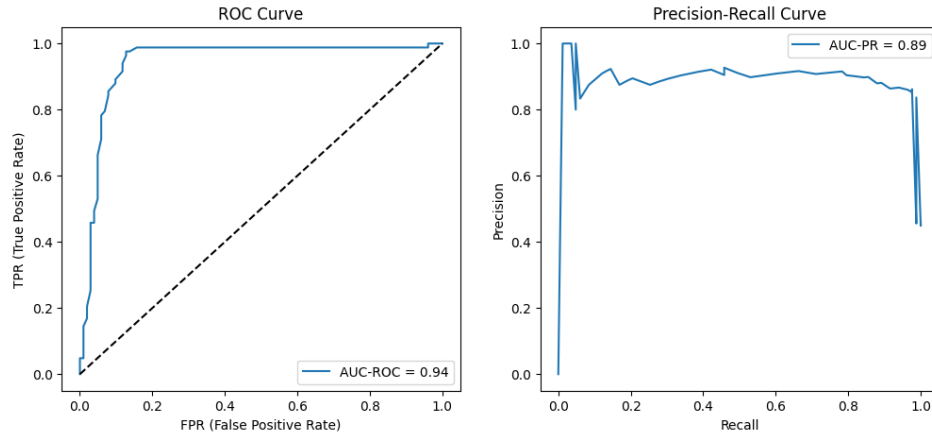


Figura 4: Curvas ROC y PR del modelo en el conjunto de datos de prueba.

A.6. Métricas de performance del ejercicio 1.4

Modelo	Accuracy	Precision	Recall	F1-score	AUC ROC	AUC PR
Sin rebalanceo	0.86	0.81	0.89	0.85	0.93	0.68
Undersampling	0.84	0.80	0.90	0.85	0.94	0.71
Oversampling duplicación	0.84	0.80	0.90	0.84	0.93	0.70
Oversampling SMOTE	0.81	0.77	0.87	0.82	0.93	0.68
Cost re-weighting	0.87	0.82	0.90	0.85	0.93	0.67

Cuadro 5: Métricas en el conjunto de validación desbalanceado, utilizando diferentes técnicas de rebalanceo.

A.7. Curvas AUC ROC y AUC PR del ejercicio 1.4

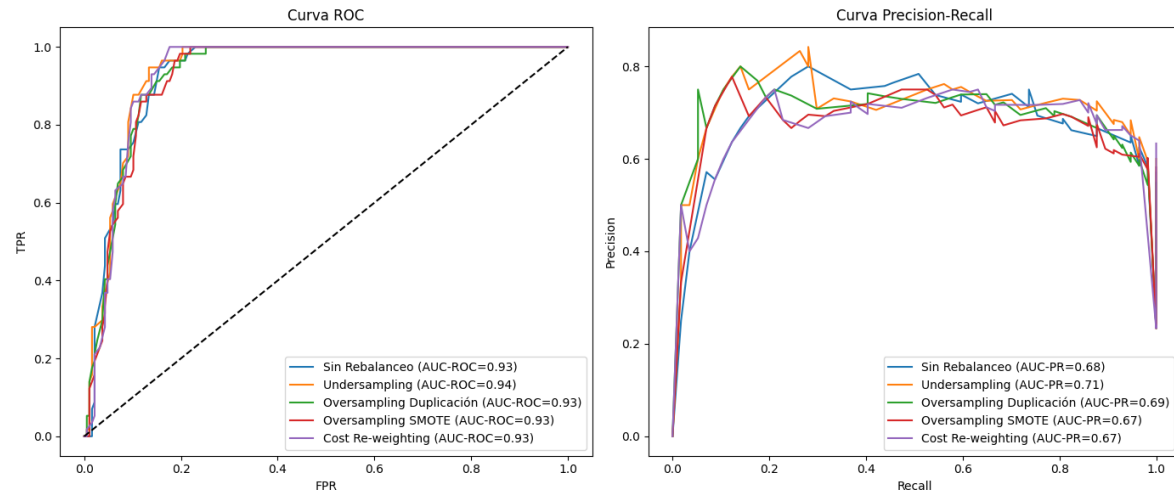


Figura 5: Curvas ROC y PR del modelo en el conjunto de datos de validación desbalanceado.

A.8. Métricas de performance del ejercicio 1.5

Modelo	Accuracy	Precision	Recall	F1-score	AUC ROC	AUC PR
Sin rebalanceo	0.93	0.89	0.94	0.91	0.95	0.78
Undersampling	0.86	0.82	0.91	0.86	0.95	0.77
Oversampling duplicación	0.87	0.83	0.92	0.87	0.95	0.77
Oversampling SMOTE	0.84	0.80	0.89	0.85	0.96	0.79
Cost re-weighting	0.93	0.89	0.95	0.92	0.96	0.78

Cuadro 6: Métricas en el conjunto de prueba desbalanceado, utilizando diferentes técnicas de rebalanceo.

A.9. Información mutua con war_class

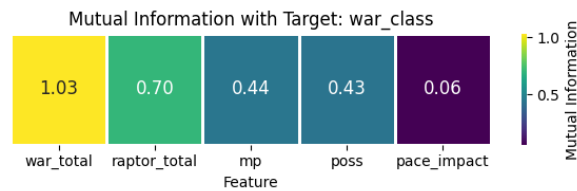


Figura 6: Información mutua entre las variables predictoras y la variable objetivo `war_class`.

Mediante un análisis de la figura es posible determinar que `war_total` posee una alta correlación con `war_class`, lo que sugiere que podría ser redundante y generar ruido en el modelo. Por lo tanto, se decidió eliminarla del conjunto de datos.

A.10. F1-score vs λ del ejercicio 2.3

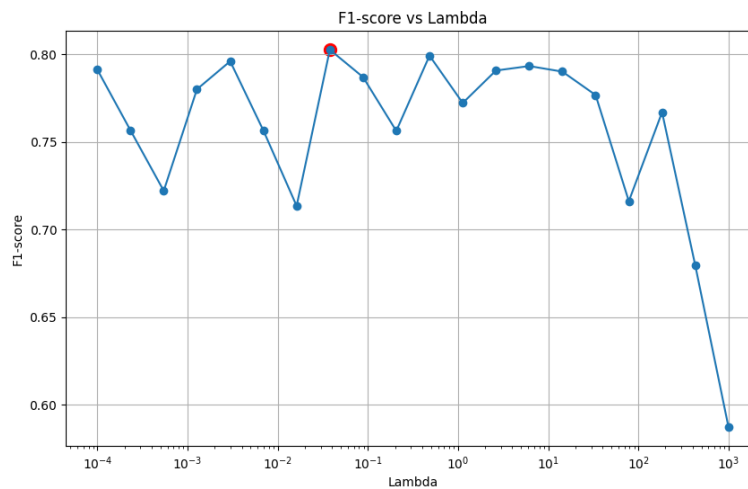


Figura 7: Evolución de λ vs F1-score utilizando un único barrido.

Producto de su estudio se determinó que el valor óptimo de λ es 0.0379, ya que a partir de este valor la métrica comienza a decrecer. Resultando ser el punto donde se maximiza el *F1-score*.

A.11. Métricas de performance del ejercicio 2.3

Modelo	Accuracy	Precision	Recall	F1-score	AUC ROC	AUC PR
LDA	0.89	0.90	0.89	0.89	0.96	0.91
LogReg Softmax L2	0.88	0.88	0.88	0.88	0.95	0.90
Random Forest	0.94	0.94	0.94	0.94	0.99	0.98

Cuadro 7: Métricas en el conjunto de validación, utilizando modelos de clasificación multiclase.

A.12. Curvas AUC ROC y AUC PR del ejercicio 2.3

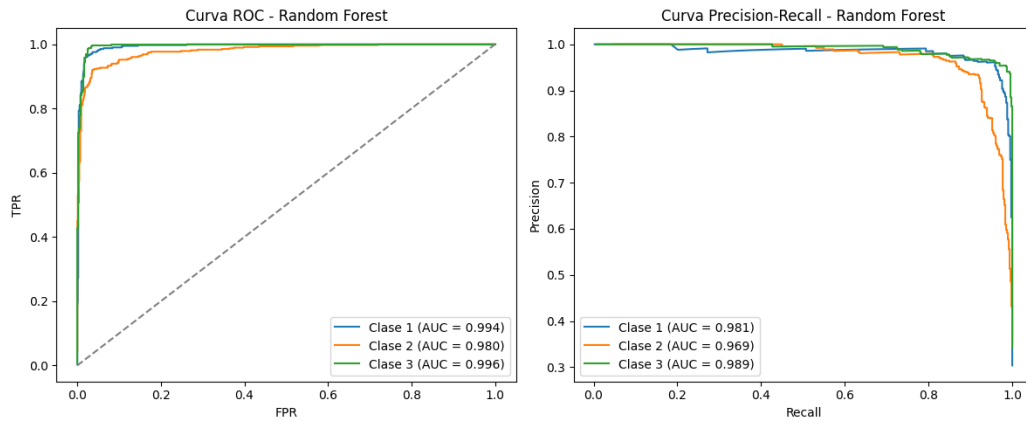


Figura 8: Curvas AUC ROC y AUC PR para el modelo Random Forest.

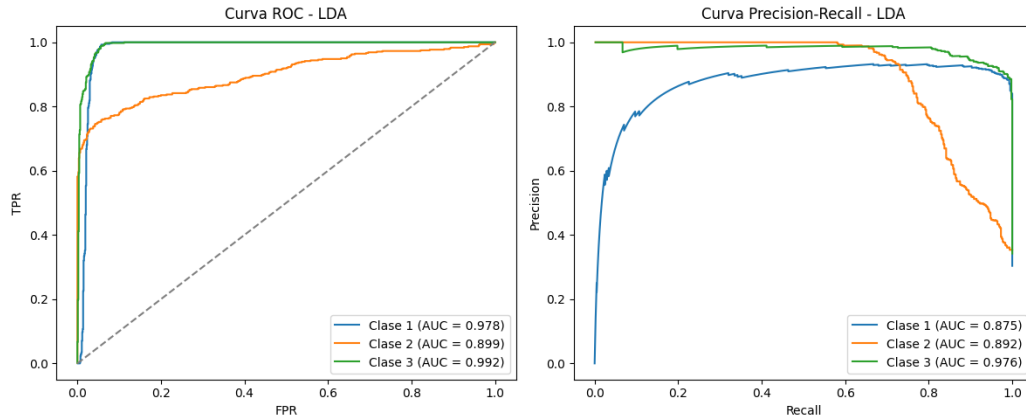


Figura 9: Curvas AUC ROC y AUC PR para el modelo LDA.

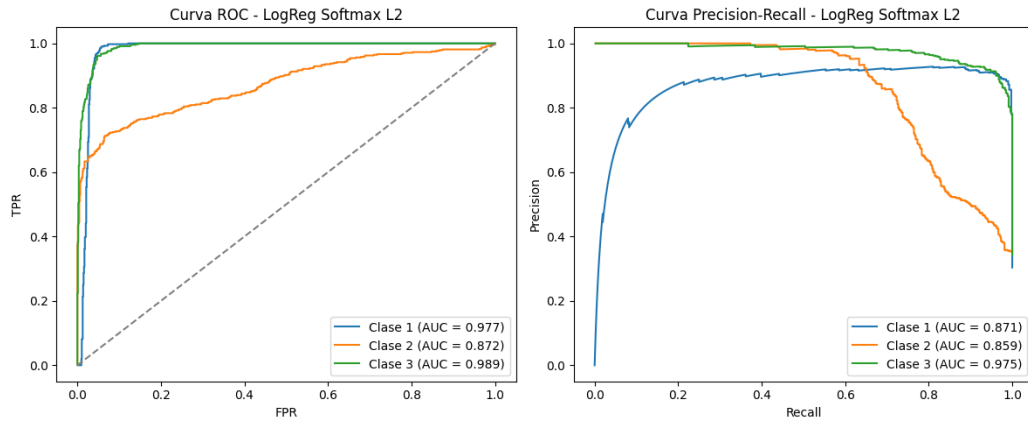


Figura 10: Curvas AUC ROC y AUC PR para el modelo de regresión logística multiclase.

A.13. Métricas de performance del ejercicio 2.4

Modelo	Accuracy	Precision	Recall	F1-score	AUC ROC	AUC PR
LDA	0.89	0.90	0.90	0.89	0.95	0.88
LogReg Softmax L2	0.88	0.88	0.89	0.88	0.94	0.88
Random Forest	0.94	0.94	0.95	0.94	0.99	0.98

Cuadro 8: Métricas en el conjunto de prueba, utilizando diferentes modelos de clasificación multiclase.